

Airflow Helper Functions

Step-by-Step Explanation

A beginner-friendly guide to timezone settings, default DAG arguments, task creation, path handling, environment variables, and subprocess execution

Purpose of this file: It acts as a shared utility module that multiple Airflow DAGs can import. This prevents repeated code and keeps timezone, retry, task, and script-execution behaviour consistent.

Prepared as a learning and revision guide

1. What the complete file does

This Python file contains reusable helper functions for Apache Airflow DAGs. Instead of copying the same setup code into every DAG, a DAG can import these functions and use them when required.

1. Returns the Australia/Sydney timezone.
2. Creates a standard dictionary of default Airflow DAG arguments.
3. Creates an EmptyOperator task that can be used as a start or placeholder task.
4. Finds a Python script inside a shared scripts folder and runs it as a separate process.
5. Adds the DAG project folders to PYTHONPATH so imported project modules can be located.
6. Stops the Airflow task with a clear error when the Python script fails.

Expected project structure

```
learning_airflow/
|-- dags/
|   |-- helpers/
|   |   |-- airflow_helpers.py    <- this file
|   |-- example_dag.py
|-- scripts/
|   |-- load_data.py
|-- sql/
|   |-- transformation.sql
```

Because the helper file is assumed to be inside a subfolder under the DAG root, `Path(__file__).resolve().parents[1]` moves up two levels to find the main ``dags`` folder.

Complete code

```
import os
import subprocess
import sys
from datetime import datetime, timedelta
from pathlib import Path
import pendulum

def get_local_timezone():
    return pendulum.timezone("Australia/Sydney")

def get_default_args(owner: str = "learner") -> dict:
    tz = get_local_timezone()
    return {
        "owner": owner,
        "depends_on_past": False,
        "start_date": datetime(2026, 1, 1, tzinfo=tz),
        "retries": 0,
        "retry_delay": timedelta(minutes=5),
    }

def create_start_sensor(dag, hour: int = 7, minute: int = 0):
    from airflow.operators.empty import EmptyOperator

    return EmptyOperator(task_id=f"wait_until_{hour:02d}_{minute:02d}", dag=dag)

def run_python_script(script_name: str):
    dag_root = Path(__file__).resolve().parents[1]
    script_dir = dag_root / "scripts"
    script_path = script_dir / script_name
    env = os.environ.copy()
    existing_pythonpath = env.get("PYTHONPATH", "")
    extra_paths = os.pathsep.join([str(dag_root), str(script_dir)])
    env["PYTHONPATH"] = extra_paths + (os.pathsep + existing_pythonpath if existing_pythonpath else "")
    command = [sys.executable, str(script_path)]
    print(f"Running script command: {command}")
    print(f"Working directory: {dag_root}")
    print(f"PYPATH: {env['PYTHONPATH']}")

    try:
        subprocess.run(command, check=True, cwd=str(dag_root), env=env)
    except subprocess.CalledProcessError as exc:
        raise RuntimeError(f"Script failed: {script_name} exited with code {exc.returncode}") from exc
```

2. Import statements

The import section loads standard Python modules and the Pendulum library. Each import supports a different part of the helper file.

Code	Simple meaning	Why it is needed
import os	Provides access to operating-system settings.	Used to copy environment variables and to obtain the correct path separator for Windows or Linux.
import subprocess	Runs another program or Python script.	Used to execute a script as a separate process and detect whether it succeeded or failed.
import sys	Provides information about the current Python runtime.	<code>`sys.executable`</code> points to the exact Python interpreter currently running Airflow.
datetime	Represents a specific date and time.	Used to define the DAG start date.
timedelta	Represents a duration.	Used to define a five-minute retry delay.
Path	Creates and combines file-system paths safely.	Avoids manually joining path strings and works across Windows and Linux.
pendulum	Provides timezone-aware date and time handling.	Airflow uses Pendulum heavily for timezone-aware scheduling.

Why use Path instead of strings?: A path such as ``dag_root / "scripts" / script_name`` is easier to read and automatically uses the correct slash style for the operating system.

3. ``get_local_timezone()``

```
def get_local_timezone():  
    return pendulum.timezone("Australia/Sydney")
```

Code	Simple meaning	Why it is needed
def get_local_timezone():	Defines a function with no input parameters.	Other functions can call it whenever they need the project timezone.
pendulum.timezone(...)	Creates a timezone object for Sydney.	Sydney changes between standard time and daylight-saving time automatically.
return ...	Sends the timezone object back to the caller.	The caller can store it in a variable such as <code>`tz`</code> .

Example result: during Australian Eastern Standard Time the offset is normally UTC+10, while during daylight-saving time it is normally UTC+11. The named timezone handles this change automatically; a fixed UTC offset would not.

4. `get\_default\_args()`

```
def get_default_args(owner: str = "learner") -> dict:
    tz = get_local_timezone()
    return {
        "owner": owner,
        "depends_on_past": False,
        "start_date": datetime(2026, 1, 1, tzinfo=tz),
        "retries": 0,
        "retry_delay": timedelta(minutes=5),
    }
```

Function definition

Code	Simple meaning	Why it is needed
owner: str	The owner should be supplied as text.	The type hint helps humans and development tools understand the expected input.
= "learner"	Uses `learner` when no owner is supplied.	`get_default_args()` works immediately, while `get_default_args("Mohan")` overrides it.
-> dict	The function is expected to return a dictionary.	Airflow accepts default arguments in dictionary form.

Dictionary entries

Code	Simple meaning	Why it is needed
"owner": owner	Records who owns or maintains the DAG.	Useful for organisation and responsibility; it does not automatically grant permissions.
"depends_on_past": False	A task run does not depend on the previous scheduled run succeeding.	A failed run yesterday will not automatically block today's run.
"start_date": ...	Defines the earliest logical scheduling date.	Airflow uses it with the schedule to determine eligible DAG runs; it is not a command to run immediately at that exact moment.
"retries": 0	Do not retry a failed task.	The task fails after its first unsuccessful attempt.
"retry_delay": 5 min	Wait five minutes between retries.	This setting has no practical effect while retries remains zero, but it is ready if retries is later increased.

Airflow timing note: For modern Airflow projects, `pendulum.datetime(2026, 1, 1, tz="Australia/Sydney")` is often a clearer way to construct the start date. The current code still expresses the intention: a timezone-aware start date.

5. `create\_start\_sensor()`

```
def create_start_sensor(dag, hour: int = 7, minute: int = 0):
    from airflow.operators.empty import EmptyOperator

    return EmptyOperator(task_id=f"wait_until_{hour:02d}_{minute:02d}", dag=dag)
```

Step-by-step

7. `dag` receives the DAG object to which the new task will belong.
8. `hour` defaults to 7 and `minute` defaults to 0.
9. The `EmptyOperator` is imported inside the function. This delays the Airflow-specific import until the function is called.
10. An `EmptyOperator` task is created and returned.
11. The task ID is built using the hour and minute, for example `wait\_until\_07\_00`.

Code	Simple meaning	Why it is needed
{hour:02d}	Formats the hour as two digits.	`7` becomes `07`; `12` remains `12`.
{minute:02d}	Formats the minute as two digits.	`0` becomes `00`; `30` remains `30`.
dag=dag	Explicitly attaches the operator to the supplied DAG.	Useful in code that does not rely only on a DAG context manager.

Important correction: Despite the function name, this is not a time sensor. `EmptyOperator` completes immediately when scheduled. The hour and minute only change the task ID. To truly wait until a clock time, use a suitable Airflow time sensor or design the DAG schedule to begin at that time.

What happens when called?

```
start = create_start_sensor(dag)
# Creates task_id: wait_until_07_00
# The task performs no work and succeeds immediately when executed.
```

6. `run\_python\_script()` - locating the script

```
def run_python_script(script_name: str):
    dag_root = Path(__file__).resolve().parents[1]
    script_dir = dag_root / "scripts"
    script_path = script_dir / script_name
```

Line 1: function input

`script\_name: str` means the function expects a file name such as `load\_data.py`.

Line 2: find the DAG root

Code	Simple meaning	Why it is needed
<code>__file__</code>	The path of the current helper Python file.	Gives the code a reliable starting point instead of depending on the terminal location.
<code>Path(__file__)</code>	Converts that path to a Path object.	Enables safe path operations.
<code>.resolve()</code>	Converts it to an absolute, normalised path.	Removes uncertainty from relative paths and resolves `.` or `..` components.
<code>.parents[1]</code>	Moves up two parent levels.	With `dags/helpers/airflow_helpers.py`, parent 0 is `helpers` and parent 1 is `dags`.

Example path calculation

```
Current file:
/airflow/dags/helpers/airflow_helpers.py

Path(__file__).resolve().parents[0]
-> /airflow/dags/helpers

Path(__file__).resolve().parents[1]
-> /airflow/dags          # dag_root
```

Lines 3-4: build the script path

```
script_dir = dag_root / "scripts"
script_path = script_dir / script_name

# Example:
# dag_root   = /airflow/dags
# script_dir = /airflow/dags/scripts
# script_path= /airflow/dags/scripts/load_data.py
```

Assumption: This path logic assumes the `scripts` folder is inside the calculated `dag\_root`. If your actual Composer structure is different, the path must be adjusted.

7. Preparing environment variables and PYTHONPATH

```
env = os.environ.copy()
existing_pythonpath = env.get("PYTHONPATH", "")
extra_paths = os.pathsep.join([str(dag_root), str(script_dir)])
env["PYTHONPATH"] = extra_paths + (
    os.pathsep + existing_pythonpath if existing_pythonpath else ""
)
```

Why create `env`?

`os.environ` contains environment variables available to the Airflow task, such as PATH, HOME, and possibly credentials or project settings. `.copy()` creates a separate dictionary so this function can add PYTHONPATH without changing the parent Airflow process globally.

Code	Simple meaning	Why it is needed
<code>env.get("PYTHONPATH", "")</code>	Reads the current PYTHONPATH, or returns an empty string.	Preserves any paths that were already configured.
<code>str(dag_root)</code>	Converts the Path object to text.	Environment variables must contain strings.
<code>str(script_dir)</code>	Converts the scripts path to text.	Makes modules stored in scripts importable.
<code>os.pathsep</code>	Returns the correct separator between multiple paths.	Linux uses `:`; Windows uses `;`.
<code>os.pathsep.join(...)</code>	Combines the two new paths.	Produces a valid multi-folder PYTHONPATH value.
<code>existing_pythonpath else ""</code>	Adds the previous value only when it exists.	Avoids adding an unnecessary trailing separator.

Example on Cloud Composer/Linux

```
extra_paths = "/airflow/dags:/airflow/dags/scripts"
existing_pythonpath = "/opt/project/common"

final PYTHONPATH =
"/airflow/dags:/airflow/dags/scripts:/opt/project/common"
```

Why PYTHONPATH matters

Suppose `load\_data.py` contains `from common.helpers import validate\_data`. Python must know where the `common` package is located. Adding the DAG root and scripts folder to PYTHONPATH helps the child script find project modules.

8. Building the command and writing logs

```
command = [sys.executable, str(script_path)]
print(f"Running script command: {command}")
print(f"Working directory: {dag_root}")
print(f"PYTHONPATH: {env['PYTHONPATH']}")
```

``sys.executable``

``sys.executable`` gives the full path of the Python interpreter currently running the Airflow worker. This is safer than writing only ``python``, because it uses the same environment and installed packages as the current process.

Why the command is a list

The list separates the executable from each argument. For example:

```
[
    "/opt/python/bin/python",
    "/airflow/dags/scripts/load_data.py"
]
```

Passing a list avoids fragile string parsing and is safer than constructing one shell command. No ``shell=True`` is required.

Why the print statements are useful

- They appear in the Airflow task logs.
- They show exactly which script and interpreter were used.
- They confirm the working directory.
- They help diagnose import errors by displaying PYTHONPATH.

9. Executing the script with ``subprocess.run()``

```
try:
    subprocess.run(
        command,
        check=True,
        cwd=str(dag_root),
        env=env
    )
except subprocess.CalledProcessError as exc:
    raise RuntimeError(
        f"Script failed: {script_name} exited with code {exc.returncode}"
    ) from exc
```

Code	Simple meaning	Why it is needed
command	The Python interpreter and script path.	Tells the operating system what to execute.
check=True	Raise an exception when the script returns a non-zero exit code.	Allows Airflow to mark the task as failed instead of silently continuing.
cwd=str(dag_root)	Run with the DAG root as the current working directory.	Relative file references inside the child script are resolved from this folder.
env=env	Pass the copied and updated environment variables.	The child process receives the custom PYTHONPATH and existing Airflow environment.

10. Success and failure behaviour

When the script succeeds

12. The child Python script finishes with exit code 0.
13. `subprocess.run()` returns normally.
14. The helper function ends without raising an exception.
15. The Airflow task can be marked successful.

When the script fails

16. The child Python script finishes with a non-zero exit code, such as 1.
17. Because `check=True`, Python raises `subprocess.CalledProcessError`.
18. The `except` block catches that specific exception.
19. A clearer `RuntimeError` is raised with the script name and exit code.
20. Airflow sees the exception and marks the task as failed.

```
Example Airflow error:  
RuntimeError: Script failed: load_data.py exited with code 1
```

`raise ... from exc` preserves the original exception as the underlying cause. This gives a cleaner high-level message while retaining useful debugging information in the traceback.

11. End-to-end execution flow

```
Airflow Python task starts  
|  
v  
run_python_script("load_data.py")  
|  
v  
Find dag_root and scripts folder  
|  
v  
Build /dags/scripts/load_data.py  
|  
v  
Copy environment and update PYTHONPATH  
|  
v  
Use current Python interpreter  
|  
v  
Run child script with subprocess  
|  
+---- exit code 0 ----> Airflow task succeeds  
|  
`---- non-zero -----> RuntimeError -> Airflow task fails
```

12. Example of using these helpers in an Airflow DAG

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from helpers.airflow_helpers import (
    get_default_args,
    create_start_sensor,
    run_python_script,
)

with DAG(
    dag_id="daily_data_pipeline",
    default_args=get_default_args(owner="data_team"),
    schedule="0 7 * * *",
    catchup=False,
) as dag:

    start = create_start_sensor(dag, hour=7, minute=0)

    load_data = PythonOperator(
        task_id="load_data",
        python_callable=run_python_script,
        op_args=["load_data.py"],
    )

    start >> load_data
```

What happens here?

21. The DAG imports the reusable helper functions.
22. `get_default_args(owner="data_team")` creates the standard default arguments.
23. The DAG schedule `0 7 * * *` means the DAG is scheduled daily at 7:00 AM in the DAG timezone configuration.
24. `create_start_sensor()` creates a no-work start task named `wait_until_07_00`.
25. The PythonOperator calls `run_python_script` and passes `load_data.py` as its argument.
26. The dependency `start >> load_data` means the load task begins after the start task succeeds.

Scheduling recommendation: Use the DAG schedule to control when the pipeline starts. Do not rely on the current EmptyOperator helper to wait for a particular clock time.

13. Practical checks and possible improvements

Code	Simple meaning	Why it is needed
Check file exists	Test <code>script_path.exists()</code> before running.	Produces an immediate, clear error when the supplied file name is wrong.
Use a true sensor or rename	Rename to <code>create_start_task</code> , or use a time sensor.	Makes the function name accurately describe its behaviour.
Set retries intentionally	Increase retries only for temporary failures.	Retrying will not fix permanent coding errors but can help with transient API or network failures.
Validate input	Reject paths outside the scripts directory.	Prevents unexpected path traversal such as <code>../other_file.py</code> .
Capture output if needed	Use <code>capture_output=True</code> , <code>text=True</code> selectively.	Allows programmatic handling of stdout/stderr, although normal inherited logs are often easier in Airflow.
Use TaskFlow where suitable	Airflow <code>@task</code> can simplify Python tasks.	Reduces boilerplate for many modern DAGs.

14. Safer improved version of `run\_python\_script()`

The following version adds a file-existence check and confirms that the final path remains inside the scripts folder.

```
def run_python_script(script_name: str):
    dag_root = Path(__file__).resolve().parents[1]
    script_dir = (dag_root / "scripts").resolve()
    script_path = (script_dir / script_name).resolve()

    if script_dir not in script_path.parents:
        raise ValueError(f"Invalid script path: {script_name}")

    if not script_path.is_file():
        raise FileNotFoundError(f"Script not found: {script_path}")

    env = os.environ.copy()
    existing_pythonpath = env.get("PYTHONPATH", "")
    extra_paths = os.pathsep.join([str(dag_root), str(script_dir)])
    env["PYTHONPATH"] = extra_paths + (
        os.pathsep + existing_pythonpath
        if existing_pythonpath
        else ""
    )

    command = [sys.executable, str(script_path)]

    try:
        subprocess.run(
            command,
            check=True,
            cwd=str(dag_root),
            env=env,
        )
    except subprocess.CalledProcessError as exc:
        raise RuntimeError(
            f"Script failed: {script_name} exited with code {exc.returncode}"
        ) from exc
```

This improved version is optional. Your original code already demonstrates the key Airflow pattern: determine the path, prepare the environment, execute the script, and propagate failures to Airflow.

15. Easy interview or revision summary

One-paragraph explanation: This helper module centralises common Airflow configuration. It returns the Sydney timezone, supplies consistent default DAG arguments, creates an EmptyOperator start task, and provides a reusable function to run Python scripts from a shared scripts folder. The script runner builds reliable file paths with pathlib, preserves the Airflow environment, extends PYTHONPATH for project imports, uses the same Python interpreter as Airflow, and raises an error when the child script fails so the Airflow task is correctly marked as failed.

Key points to remember

- `get\_local\_timezone()` gives timezone awareness.
- `get\_default\_args()` prevents repeated DAG configuration.
- `EmptyOperator` is a placeholder, not a clock-time sensor.
- `\_\_file\_\_` makes path discovery independent of where the command was launched.
- `PYTHONPATH` helps the child script import project modules.
- `sys.executable` uses the same Python environment as Airflow.
- `check=True` converts a non-zero child exit code into a Python exception.
- Raising an exception is how the helper tells Airflow that the task failed.